MOBILE SECURITY PACKAGE
ASTRA
SPACE THEMED CTF

Level - 1 - SIGNAL ORIGIN

We launch the jadx-gui and then drag and drop the apk and open the level1 activity.
In the decompiled code we could find

```
@Override // com.astra.blacksite.core.BaseLevelActivity
protected boolean validateFlagInput(String input) {
    String assembled = getString(R.string.cfg_relay_prefix) + getString(R.string.cfg_station_token) + getString(R.string.cfg_relay_suffix);
    return assembled.equals(input);
}
```

The flag is not generated dynamically and It is constructed from 3 string resources. If we look into strings.xml, we could find

```
47      <string name="cfg_relay_prefix">ASTRA{5t4t1c</string>
48      <string name="cfg_relay_suffix">_50_3asy}</string>
49      <string name="cfg_session_key">NzQ4M2FlYmM5ZjQx</string>
50      <string name="cfg_station_token">_r3s0urc3_n0t</string>
51      <string name="cfg_token_type">BEARER_HMAC_SHA256</string>
```

Through this we could find that the flag of the first level is **ASTRA{5t4t1c_r3s0urc3_n0t_50_3asy}**

Level - 2 - FREQUENCY MASK

Now if we look into the decompiled code of the level 2 activity we find

```
private String decode() {
    byte[] out = new byte[ENC.length];
    int i = 0;
    while (true) {
        byte[] bArr = ENC;
        if (i < bArr.length) {
            out[i] = (byte) (bArr[i] ^ XOR_KEY);
            i++;
        } else {
            return new String(out);
        }
    }
}

@Override // com.astra.blacksite.core.BaseLevelActivity
protected boolean validateFlagInput(String input) {
    return decode().equals(input);
}
```

ValidateFlagInput() says that it compares input with the result of decode().So our goal is to reverse the decode() function to recover the flag.
In the decode() function we understand the decoding logic which is

out[i] = (byte) (bArr[i] ^ XOR_KEY);

```
public class Level02Activity extends BaseLevelActivity {
    private static final byte[] ENC = {15, 29, 26, 28, 15, 53, 54, 126, 60, 17, Base64.padSymbol, 58,
    private static final byte XOR_KEY = 78;
```

A byte array (ENC) stores encoded data Each byte is transformed using XOR operation with XOR_KEY as 78.
Through this We can get the flag

```
enc =
[15,29,26,28,15,53,54,126,60,17,61,58,60,125,122,35,17,42,125,45,126,42,125,
42,17,126,122,124,51]
key = 78

decoded = ''.join(chr(b ^ key) for b in enc)
print(decoded)
```

And the flag is **ASTRA{x0r_str34m_d3c0d3d_042}**

LEVEL-3 SIGNAL ORIGIN

Now if we look into the decompiled code of the level 2 activity we find

```
@Override // com.astra.blacksite.core.BaseLevelActivity
protected boolean validateFlagInput(String input) {
    try {
        return nativeValidate(input);
    } catch (UnsatisfiedLinkError e) {
        return sha256(input).equals("3d33e10623252643b4c70bf30b0b2d5e3a4b5c6d7e8f9a0b1c2d3e4f50617283'
    }
}
```

The actual validation happens in nativeValidate(input) .This function is not implemented in Java .It exists inside a native library (libsignal.so).
As hinted: “The identifier does not exist in the Java or Kotlin layer at all”

```
/* JADX INFO: loaded from: classes4.dex */
public class Level03Activity extends BaseLevelActivity {
    private native String nativeGetStatus();

    private native boolean nativeValidate(String str);

    static {
        try {
            System.loadLibrary("signal");
        } catch (UnsatisfiedLinkError e) {
        }
    }
```

From code: **System.loadLibrary("signal");**
This corresponds to: libsignal.so

Now lets extract and inspect the binary to get the flag. We will use apktool for extracting the apk file. This can also be done using unzip.

```
PS C:\COLLEGE\SEM8\Mobile Security\Package> apktool .\app-debug.apk
I: Using Apktool 3.0.1 on app-debug.apk with 4 threads
I: Loading resource table...
I: Baksmaling classes6.dex...
I: Baksmaling classes5.dex...
I: Baksmaling classes4.dex...
I: Baksmaling classes7.dex...
I: Baksmaling classes3.dex...
I: Baksmaling classes2.dex...
I: Baksmaling classes8.dex...
I: Baksmaling classes.dex...
I: Decoding value resources...
I: Loading resource table from file: C:\Users\karth\AppData\Local\apktool\framework\1.apk
I: Decoding file resources...
I: Generating values XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Copying original files...
I: Copying assets...
I: Copying lib...
I: Copying unknown files...
Press any key to continue . . .
PS C:\COLLEGE\SEM8\Mobile Security\Package> cd .\app-debug\
PS C:\COLLEGE\SEM8\Mobile Security\Package\app-debug> ls


    Directory: C:\COLLEGE\SEM8\Mobile Security\Package\app-debug


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        06-04-2026     23:14                assets
d-----        06-04-2026     23:14                lib
d-----        06-04-2026     23:14                original
d-----        06-04-2026     23:14                res
d-----        06-04-2026     23:14                smali
d-----        06-04-2026     23:13                smali_classes2
d-----        06-04-2026     23:13                smali_classes3
d-----        06-04-2026     23:13                smali_classes4
d-----        06-04-2026     23:13                smali_classes5
d-----        06-04-2026     23:13                smali_classes6
d-----        06-04-2026     23:13                smali_classes7
d-----        06-04-2026     23:13                smali_classes8
d-----        06-04-2026     23:14                unknown
-a----        06-04-2026     23:14           4368 AndroidManifest.xml
-a----        06-04-2026     23:14           2279 apktool.yml


PS C:\COLLEGE\SEM8\Mobile Security\Package\app-debug> cd lib
PS C:\COLLEGE\SEM8\Mobile Security\Package\app-debug\lib> ls


    Directory: C:\COLLEGE\SEM8\Mobile Security\Package\app-debug\lib


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        06-04-2026     23:14                arm64-v8a
d-----        06-04-2026     23:14                x86_64


PS C:\COLLEGE\SEM8\Mobile Security\Package\app-debug\lib> cd .\arm64-v8a\
```

```
PS C:\COLLEGE\SEM8\Mobile Security\Package\app-debug\lib\arm64-v8a> ls


    Directory: C:\COLLEGE\SEM8\Mobile Security\Package\app-debug\lib\arm64-v8a


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        06-04-2026     23:14          13344 libsignal.so


PS C:\COLLEGE\SEM8\Mobile Security\Package\app-debug\lib\arm64-v8a> strings .\libsignal.so | findstr ASTRA
ASTRA{k3rn3l_tz_s3l1nux_d4rk_h0r
ASTRA{try_harder_go_Analyze_the_VM}
ASTRA_NATIVE
ASTRA{n4t1v3_5ymb0l_r34d_fr0m_s0}
ASTRA{ch41n_3xpl01t_nex_us_4ss3mbl3d}
```

Here we find various flags here in which few are fake. On Trying all the possible flags we find the right one.
The correct flag is **ASTRA{n4t1v3_5ymb0l_r34d_fr0m_s0}**

LEVEL - 4 - ENCRYPTED ORBIT

```java
private byte[] deriveKey() {
    try {
        String raw = Build.BOARD + ":" + SALT_A + SALT_B;
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest(raw.getBytes(StandardCharsets.UTF_8));
        return Arrays.copyOf(hash, 16);
    } catch (Exception e) {
        return new byte[16];
    }
}
```

From this we can observe that AES key is derived from: **Build.BOARD + ":" + "ASTRA_K3Y" + "_S4LT_L04".** Here SHA-256 is applied and first 16 bytes is the AES key.

```java
private void ensurePayloadStored() {
    SharedPreferences prefs = getSharedPreferences(PREFS_NAME, 0);
    if (prefs.contains(KEY_PAYLOAD)) {
        return;
    }
    try {
        String flag = generateFlag();
        byte[] key = deriveKey();
        Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
        byte[] iv = new byte[12];
        new SecureRandom().nextBytes(iv);
        cipher.init(1, new SecretKeySpec(key, "AES"), new GCMParameterSpec(128, iv));
        byte[] enc = cipher.doFinal(flag.getBytes(StandardCharsets.UTF_8));
        byte[] combined = new byte[iv.length + enc.length];
        System.arraycopy(iv, 0, combined, 0, iv.length);
        System.arraycopy(enc, 0, combined, iv.length, enc.length);
        prefs.edit().putString(KEY_PAYLOAD, bytesToHex(combined)).apply();
        Arrays.fill(key, (byte) 0);
    } catch (Exception e) {
        Log.e("L04", "Encryption failed", e);
    }
}
```

The flag is **generated at runtime and encrypted using AES-GCM with a randomly generated IV**, then stored as IV + ciphertext in SharedPreferences(sectore_telemetry.xml).

The **encryption key is derived dynamically (**deriveKey()**) and never stored**, so it must be captured during execution using frida hook.

```
Java.perform(function() {

    var L04 = Java.use("com.astra.blacksite.levels.Level04Activity");

    L04.deriveKey.implementation = function() {

        var key = this.deriveKey();

        var hex = "";
        for (var i = 0; i < key.length; i++) {
            hex += ("0" + (key[i] & 0xFF).toString(16)).slice(-2);
        }

        console.log("[+] AES KEY:", hex);
        return key;
    };
});
```

Then we have to run **frida -U -l hook_l04.js com.astra.blacksite.debug**
Now if we open Level 4 in the app and submit some random value in the submission point, we could see the AES key like:
[+] AES KEY: c1c8bb65112de224942770d3a63d664d
Next we have to pull the encrypted payload:

```
C:\Users\karth>adb shell
emu64xa:/ # cd /data/data/com.astra.blacksite.debug/shared_prefs
emu64xa:/data/data/com.astra.blacksite.debug/shared_prefs # ls
pref_sector_01.xml  pref_sector_02.xml  pref_sector_03.xml  sector_telemetry.xml
at sector_telemetry.xml                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="beacon_payload">ce661065571fc54c0066b6e03aa820acb80948db1da9b5c3e87511026488e6c374bf02c8294864d752821530c87d83a37742877e7c59da</string>
</map>
emu64xa:/data/data/com.astra.blacksite.debug/shared_prefs #
```

Here this value will be the combined=iv+encrypted_data.

Now we have to decrypt using python.
The AES_GCM Decrytion script is:

```
from Crypto.Cipher import AES


# Key from Frida
key = bytes.fromhex("c1c8bb65112de224942770d3a63d664d")


# Payload from SharedPreferences
```

```
combined =
bytes.fromhex("ce661065571fc54c0066b6e03aa820acb80948db1da9b5c3e87511026488e
6c374bf02c8294864d752821530c87d83a37742877e7c59da")

# Split
iv = combined[:12]
enc = combined[12:]

# Separate ciphertext and tag
ciphertext = enc[:-16]
tag = enc[-16:]

# Decrypt
cipher = AES.new(key, AES.MODE_GCM, nonce=iv)
plain = cipher.decrypt_and_verify(ciphertext, tag)

print(plain.decode())
```

From this this flag would be **ASTRA{56fed1042f1a6d2e_dyn}**

<u>LEVEL - 05 GHOST SCHEMA</u>

This challenge distributes the flag across **two independent layers**:

- **Local SQLite Database** → provides partial data (`code`)
- **Native Library (`libsignal.so`)** → provides hidden components (`index`, `secret`)

```java
protected boolean validateFlagInput(String input) {
    try {
        String code = this.dbHelper.getCode();
        String secret = NativeBridge.getSecret();
        String keyMaterial = code + secret;
        String expected = sha256("ASTRA{" + keyMaterial + "_FLAG}")
        return sha256(input).equals(expected);
    } catch (Exception e) {
        return false;
    }
}
```

The correct flag format is **ASTRA{code + secret + "_FLAG"}.**

Next we have to extract code from SQLite Database

From DBHelper the database schema is:

```
@Override // android.database.sqlite.SQLiteOpenHelper
public void onCreate(SQLiteDatabase db) {
    db.execSQL("CREATE TABLE missions(id INTEGER, name TEXT, code TEXT)
    db.execSQL("INSERT INTO missions VALUES (1,'relay','X1A2')");
    db.execSQL("INSERT INTO missions VALUES (2,'probe','B7F3')");
    db.execSQL("INSERT INTO missions VALUES (3,'ghost','Z9Q1')");
}

@Override // android.database.sqlite.SQLiteOpenHelper
public void onUpgrade(SQLiteDatabase db, int oldV, int newV) {
}

public String getCode() {
    SQLiteDatabase db = getReadableDatabase();
    Cursor c = db.rawQuery("SELECT code FROM missions", null);
    int index = NativeBridge.getIndex();
    c.moveToPosition(index);
    return c.getString(0);
}
```

We could find the code in mission table but we have to decide upon the code.For that getCode() function is been used.

The code is selected using:

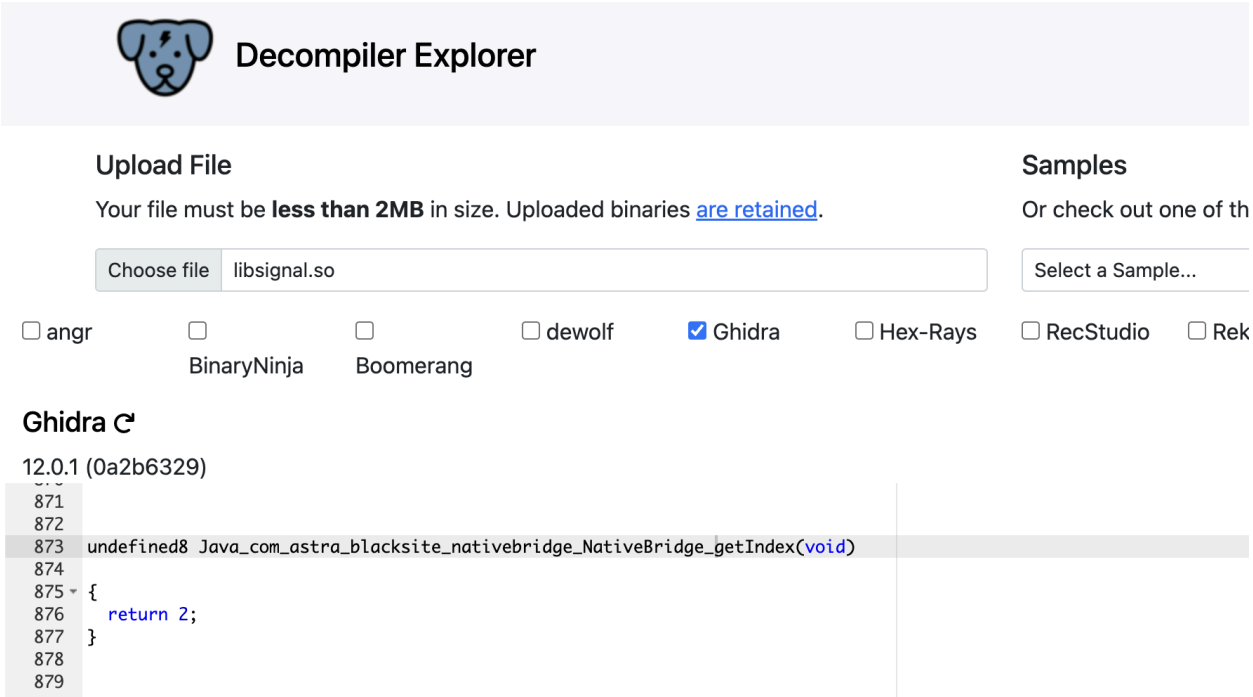
int index = NativeBridge.getIndex();
c.moveToPosition(index);

👉 So we must determine `getIndex()` from native code.

```
/* JADX INFO: loaded from: classes4.dex */
public class NativeBridge {
    public static native int getIndex();

    public static native String getSecret();

    static {
        System.loadLibrary("signal");
    }
}
```

So the index will be present in the decompiled version of libsignal.so. We find the libsignal.so file similar to that in level 3 and upload the libsignal.so into ghidra and need to find the return value of

the getIndex() function.

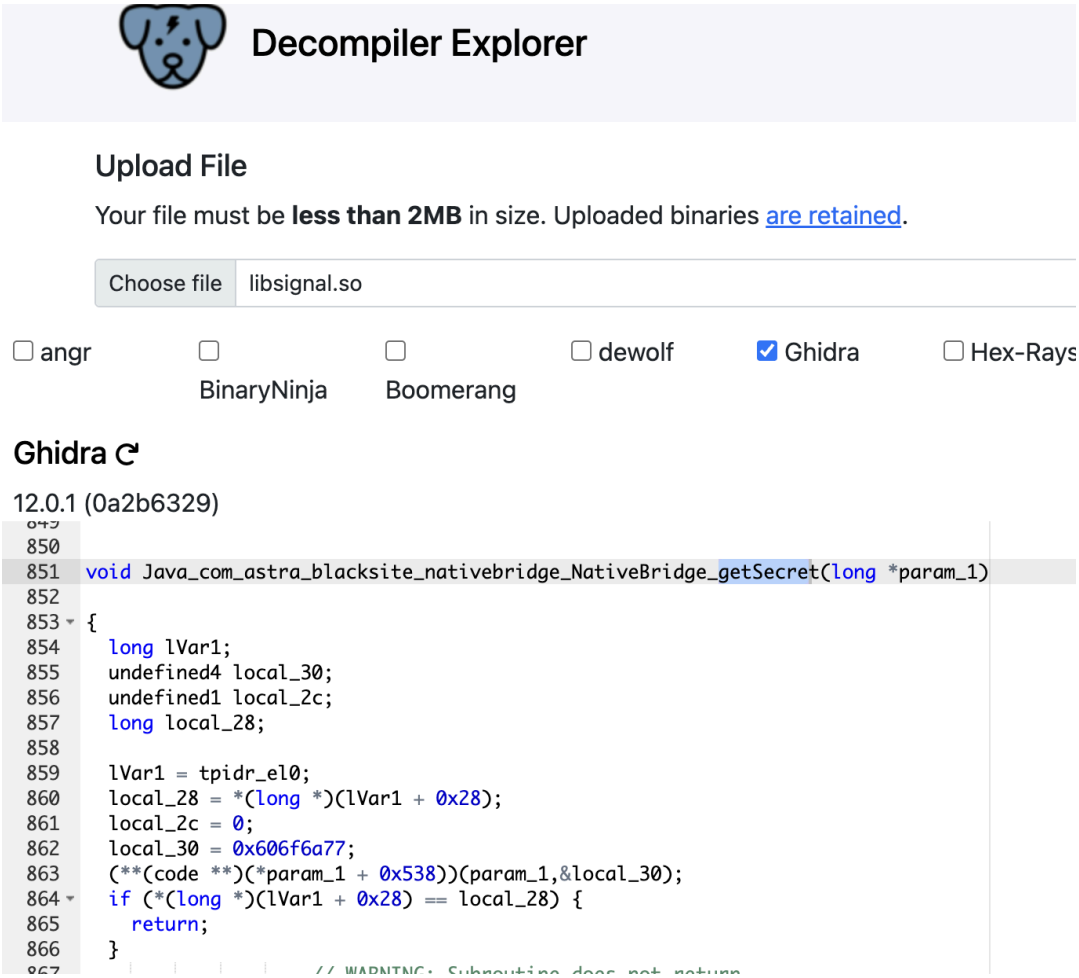


From this we can find the index is 2 and the code is “Z9Q1” as this is in the 2nd index in the missions table.

Next is we need to find the secret.

```
public class NativeBridge {
    public static native int getIndex();

    public static native String getSecret();

    static {
        System.loadLibrary("signal");
    }
}
```

The secret is also present in the decompiled version of [libsignal.so]



From this we decode the value

0x606f6a77 → little endian → 77 6a 6f 60

ASCII:

77 → w
6a → j
6f → o
60 → `
Final secret: **wjo`**

The final secret here is wjo`

Next we have to construct the flag

code = Z9Q1
secret = wjo`

So the flag is **ASTRA{Z9Q1wjo`_FLAG}**

<u>LEVEL-06 - SIGNAL ORIGIN</u>

The objective is to bypass runtime security mechanisms and extract a dynamically generated flag using instrumentation techniques.

```java
@Override // com.astra.blacksite.core.BaseLevelActivity
protected boolean validateFlagInput(String input) {
    if (!this.flagReleased || this.liveFlagBytes == null) {
        return false;
    }
    return new String(this.liveFlagBytes).equals(input);
}

@Override // com.astra.blacksite.core.BaseLevelActivity
protected void buildChallengeWidget(LinearLayout container) {
    String status;
    String str;
    boolean rooted = isRooted();
    boolean frida = isFridaPresent();
    boolean debugger = isDebuggerAttached();
    boolean emulator = isEmulator();
    boolean z = this.flagReleased;
    if (z) {
        status = "ALL_CLEAR — SECURE CHANNEL ESTABLISHED";
    } else {
        status = "LOCKDOWN — INTEGRITY VIOLATED";
    }
    int statusColor = z ? -16711779 : -50086;
    TextView tvResult = makeCodeBlock("ROOT_BINARY   : " + fmt(!rooted) + "\nFRIDA_PROCESS : " + fmt(!
    tvResult.setTextColor(statusColor);
    View[] viewArr = new View[4];
    viewArr[0] = makeSectionLabel("INTEGRITY SCAN RESULTS");
    viewArr[1] = makeDivider();
    viewArr[2] = tvResult;
    if (this.flagReleased) {
        str = "Secure memory unlocked.";
    } else {
        str = "Access denied.";
    }
    viewArr[3] = makeBodyText(str);
    container.addView(makeCard(viewArr));
}
```

The analyzed component, `Level06Activity`, implements multiple runtime integrity checks before granting access to sensitive data (the flag). The application follows a **defense-in-depth strategy**, incorporating:

- Root detection
- Frida detection
- Debugger detection
- Emulator detection

Only when all checks pass, the method `releaseClearance()` is invoked to generate the flag.

```java
public byte[] releaseClearance() {
    if (Debug.isDebuggerConnected()) {
        return null;
    }
    StackTraceElement[] stack = Thread.currentThread().getStackTrace();
    for (StackTraceElement s : stack) {
        if (s.getClassName().toLowerCase().contains("frida")) {
            return null;
        }
    }
    long t1 = System.nanoTime();
    long t2 = this.bootTime;
    String device = Build.FINGERPRINT;
    long mix = ((this.secretSeed ^ t1) ^ (t2 << 3)) ^ ((long) device.hashCode());
    int v1 = (int) Math.abs(mix % 997);
    int v2 = (int) Math.abs((mix >> 3) % 991);
    int v3 = (int) Math.abs((mix >> 5) % 983);
    String flag = "ASTRA{dyn" + v1 + "_" + v2 + "_" + v3 + "_RUNTIME_SECURE}";
    return flag.getBytes();
}
```

This code generates the flag dynamically at runtime. First, it checks if a debugger or Frida is detected—if yes, it immediately stops and returns `null`. If the environment is clean, it collects runtime values like current time (`nanoTime`), device boot time, a random secret seed, and the device fingerprint. These values are mixed together using bitwise XOR operations to create a pseudo-random number (`mix`). From this, three numbers (`v1`, `v2`, `v3`) are derived using modulo operations. Finally, these numbers are inserted into the flag format `ASTRA{dyn_v1_v2_v3_RUNTIME_SECURE}` and returned as bytes.

Now to get the flag we lauch the level and attach a frida hook to get the flag.

```java
Java.perform(function() {

  var L06 = Java.use("com.astra.blacksite.levels.Level06Activity");
  var StringCls = Java.use("java.lang.String");

  // bypass all protections
  L06.isRooted.implementation = function() { return false; };
  L06.isFridaPresent.implementation = function() { return false; };
  L06.isDebuggerAttached.implementation = function() { return false; };
  L06.isEmulator.implementation = function() { return false; };

  // extract flag
  L06.releaseClearance.implementation = function() {
    var res = this.releaseClearance();
    if (res) {
      console.log("FLAG: " + StringCls.$new(res));
    }
    return res;
  };
```

});

```
(base) ➜  Downloads frida -U -l hook-6.js -f com.astra.blacksite.debug
     ____
    / _  |   Frida 17.8.2 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Android Emulator 5554 (id=emulator-5554)
Spawned `com.astra.blacksite.debug`. Resuming main thread!
[Android Emulator 5554::com.astra.blacksite.debug ]-> FLAG: ASTRA{dyn172_876_446_RUNTIME_SECURE}
```

The flag is printed here.This flag is a dynamically generated flag.

LEVEL-07 PHANTOM SIGNAL

Techniques Used: Android RE, Native Binary Analysis, Custom Virtual Machine Reversal, Bytecode Decoding

This level introduces an **advanced reverse engineering challenge** where the flag is not stored directly in the application. Instead, it is computed through a **custom virtual machine (VM)** implemented inside a native library.

The challenge consists of three key components:

- Android Java layer (challenge logic)
- Encoded bytecode stored as an asset
- A native VM interpreter inside a shared library

The objective is to **reverse engineer the VM and reconstruct the flag generation process**.

```java
private native boolean nativeCheckL07(String str);

private native String nativeRunVM(byte[] bArr);

static {
    try {
        System.loadLibrary("signal");
    } catch (UnsatisfiedLinkError e) {
    }
}
```

```java
private byte[] loadBytecode() {
    try {
        InputStream is = getAssets().open("phantom/core.dat");
        byte[] raw = is.readAllBytes();
        is.close();
        return raw;
    } catch (Exception e) {
        return new byte[0];
    }
}
```

From this analysis we identify:

- The flag is **not stored in the Java layer**
- The program loads **bytecode from an asset file**
- Execution is delegated to a **native VM**

```
VM_ENGINE : libsignal.so :: nativeRunVM()
BYTECODE  : assets/phantom/core.dat
ENCODING  : ROLLING_XOR (start=0x37, step=+3)
OPCODE_SET: PROPRIETARY
```

This confirms that the challenge relies on **native VM interpretation of encoded bytecode**.

Next we have to extract the bytecode fron assets/phantom/core.dat . The application description indicates that the bytecode uses a **Rolling XOR encoding scheme**.

Encoding parameters:

- Initial key: **0x37**
- Increment step: **+3**

A Python script was used to reverse the encoding.

```python
data = open("core.dat", "rb").read()

key = 0x37

decoded = bytearray()

for b in data:

    decoded.append(b ^ key)
```

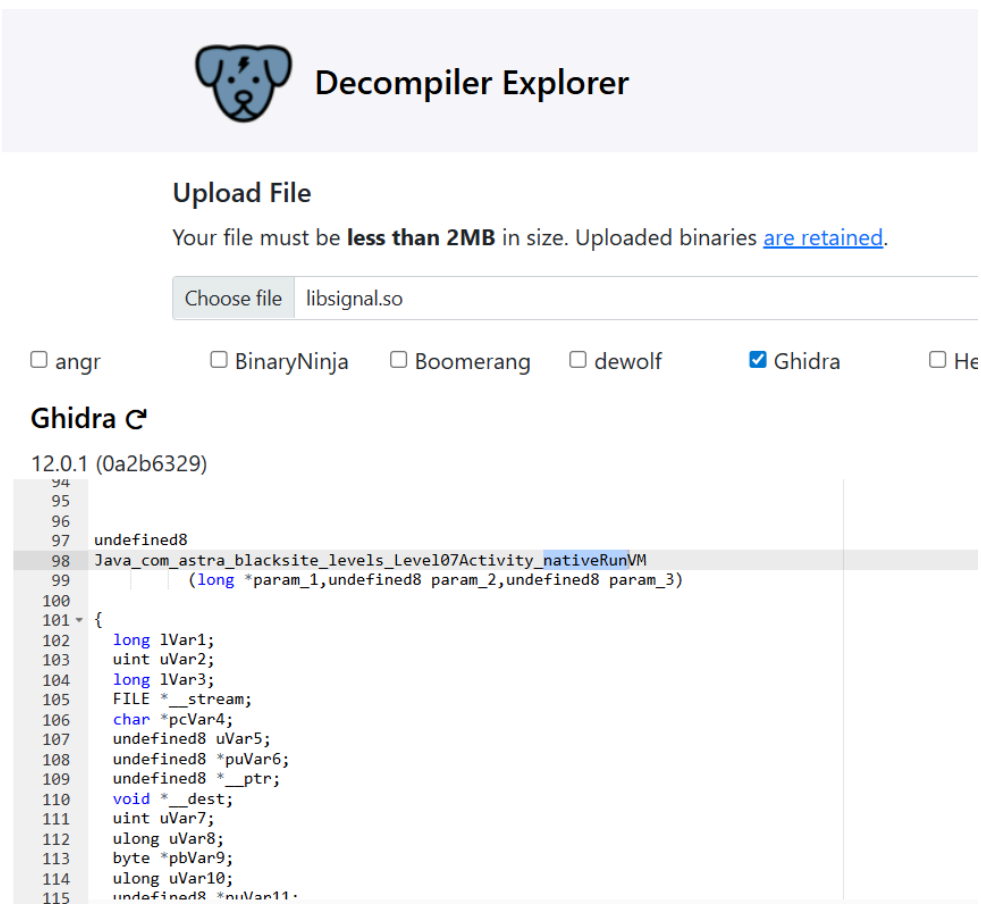
```
    key = (key + 3) & 0xff

print(decoded)

open("decoded.bin", "wb").write(decoded)
```

This produces the decoded VM instruction stream.The output for the print decoded is

bytearray(b"\x01\xd3\x02\x92\x01\x8a\x02\xd9\x01\xbc\x02\xe8\x01\xbe\x02\xec\x01\x18\x02Y\x01\x86\x02\xfd\x01\x1b\x02k\x01~\x02\x16\x01\xe2\x02\xd6\x01\xe6\x02\x88\x01\xa3\x02\xd7\x01\x17\x02\'\x01a\x02\x0c\x01\xc0\x02\x9f\x01\x1a\x02L\x01\xc8\x02\x85\x01\xd2\x02\x8d\x01\xb8\x02\x88\x01\t\x02y\x01h\x02\x0b\x01\xbb\x02\x8b\x01E\x02!\x01\xc1\x02\xf2\x01\x1f\x02@\x01\xfa\x02\x88\x01\x0b\x028\x01\xb8\x02\xce\x01\x10\x02#\x01\xd3\x02\xa1\x01\t\x02z\x01\x05\x026\x01\xd4\x02\xb0\x01\xf9\x02\x84\xff")

After this, The native library should be extracted from the APK in lib/arm64-v8a/libsignal.so

The binary was analyzed using **Ghidra** and the target function is nativeRunVm()



From the VM interpreter identified in `nativeRunVM()`, the relevant instructions are:

Opcode	Meaning
`01`	PUSH byte
`02`	XOR byte
`FF`	HALT

For this challenge, the bytecode only used **PUSH and XOR operations**.

The decoded bytecode follows a repeating structure:

```
01 XX
02 YY
```

Thus every **two instructions produce one character**. The Execution rule is CHAR = PUSH_VALUE XOR XOR_VALUE.

To verify the logic, the first few instruction pairs were evaluated. Lets consider the first instruction pair 01 D3 02 92. The Operation will be **0xD3 XOR 0x92 = 0x41** which represents A in ASCII.

The same operation was applied to every instruction pair.Processing the entire byte sequence produces the final string.

Executing the XOR operation across all PUSH/XOR pairs reconstructs the full flag:

ASTRA{ph4nt0m_VM_0pc0d3_r3v3rs3d}

The final 0xFF opcode indicates the HALT instruction, marking the end of VM execution.

<u>LEVEL 08 - GHOST MEMORY</u>

This level presents a challenge where the flag is **not statically stored inside the application**. Instead, the application generates the flag **dynamically during runtime** and passes it to a native validation function.

Because the flag only exists **temporarily in memory during execution**, static analysis alone is insufficient. The correct approach is **dynamic instrumentation** using **Frida** to intercept the function call and extract the generated flag.

The APK was first analyzed using **JADX** to identify where flag validation occurs.

```
@Override // com.astra.blacksite.core.BaseLevelActivity
protected boolean validateFlagInput(String input) {
    String str;
    if (!this.windowOpen || (str = this.sessionSecret) == null) {
        return false;
    }
    return nativeValidateFromVault(input, str);
}
```

The function checks whether a runtime condition (`windowOpen`) is satisfied. The user input and a runtime value called **sessionSecret** are passed to a **native function**: nativeValidateFromVault(input, sessionSecret). This indicates that the **actual flag comparison occurs inside the native layer**.

Further analysis revealed that `sessionSecret` is generated dynamically.

```
private String buildRuntimeSecret() {
    String timeHex = String.format("%012x", Long.valueOf(System.currentTimeMillis()));
    String part2 = timeHex.substring(4, 12);
    String part3 = Build.MODEL.replace(" ", "").toLowerCase();
    if (part3.length() > 4) {
        part3 = part3.substring(0, 4);
    }
    return "ASTRA{" + part2 + part3 + "_dyn4mic}";
}
```

Inside this function the flag structure is constructed: **return "ASTRA{" + part2 + part3 + "_dyn4mic}";**

Therefore, the only moment where the complete flag exists is during the call to nativeValidateFromVault().

Since the flag exists only during runtime execution, the objective is to intercept the function call where it appears.The most efficient approach is **runtime hooking using Frida**.

A Frida script was written to hook the native validation function and print the secret argument.

```
Java.perform(function () {

  var cls = Java.use("com.astra.blacksite.levels.Level08Activity");

  cls.nativeValidateFromVault.overload(
    'java.lang.String',
    'java.lang.String'
```

```
    ).implementation = function (input, secret) {

        console.log("\n FLAG: " + secret + "\n");

        return this.nativeValidateFromVault(input, secret);
    };

});
```

The application was launched with Frida attached.

frida -U -f com.astra.blacksite.debug -l script.js --no-pause

Steps inside the application:

1. Open **Level 08**
2. Click **AUTHENTICATE**
3. Enter any value (e.g., `aaa`)
4. Click **SUBMIT**

This triggers the validation function and activates the Frida hook.

When the validation function executes, the Frida script prints the runtime secret.
FLAG: ASTRA{579e3dd6sdk__dyn4mic}